

Instance Segmentation

Instance Segmentation

- 1. Model Introduction
- 2. Instance Segmentation: Image
 - Effect Preview
- 3. Instance Segmentation: Video
 - Effect Preview
- 4. Instance Segmentation: Real-time Detection
 - 4.1. USB Camera
 - Effect Preview
 - 4.2. CSI Camera
 - Effect Preview
- Reference Materials

Use Python to demonstrate **Ultralytics**: Instance Segmentation effects on images, videos, and real-time detection.

1. Model Introduction

Instance segmentation goes a step further than object detection, involving identifying individual objects in an image and separating them from the rest of the image.

The output of an instance segmentation model is a set of masks or contours that outline each object in the image, along with class labels and confidence scores for each object. When you not only need to know the position of objects in the image but also their specific shapes, instance segmentation becomes very useful.

2. Instance Segmentation: Image

Use yolo26n-seg_ncnn_model to predict images in the ultralytics26 folder.

Enter the code folder:

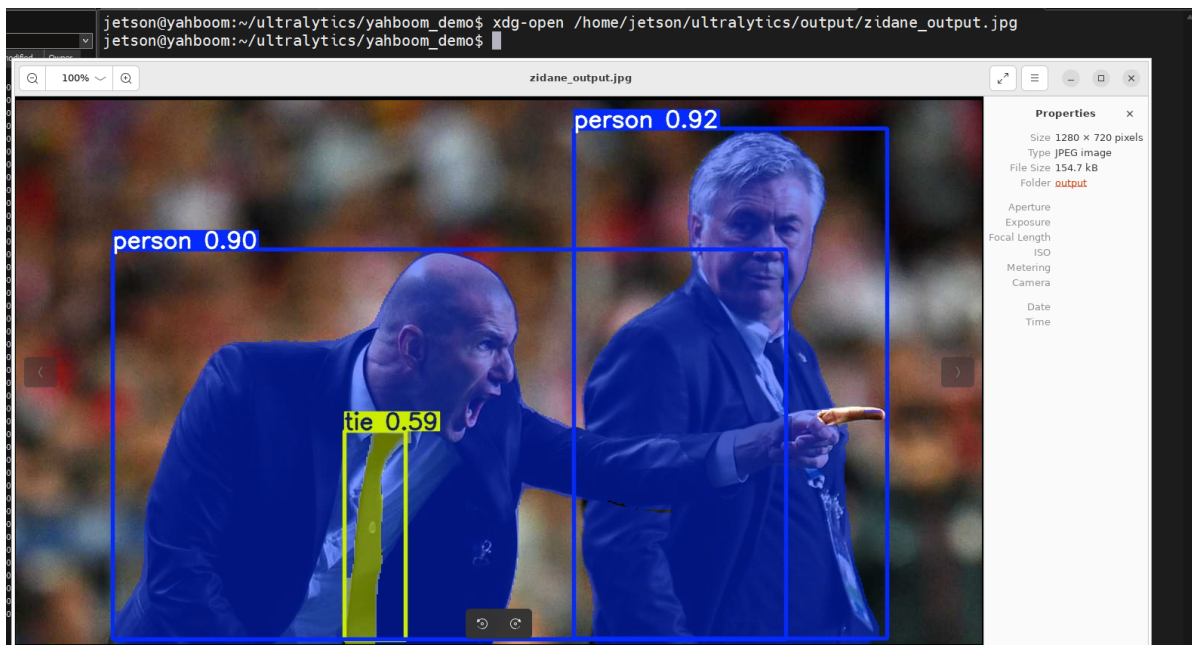
```
cd /home/jetson/ultralytics/yahboom_demo
```

Run the code:

```
python3 02.segmentation_image.py
```

Effect Preview

yolo recognition output image location: /home/jetson/ultralytics/output/



Example code:

```
from ultralytics import YOLO

# Load a model
model = YOLO("/home/jetson/ultralytics/yolo26n-seg.engine") # pretrained
yolo26n model from ultralytics

# Run batched inference on a list of images
results = model("/home/jetson/ultralytics/assets/zidane.jpg") # return a list
of Results objects

# Process results list
for result in results:
    # boxes = result.boxes # Boxes object for bounding box outputs
    masks = result.masks # Masks object for segmentation masks outputs
    # keypoints = result.keypoints # Keypoints object for pose outputs
    # probs = result.probs # Probs object for classification outputs
    # obb = result.obb # Oriented boxes object for OBB outputs
    result.show() # display to screen
    result.save(filename="/home/jetson/ultralytics/output/zidane_output.jpg") #
    save to disk
```

3. Instance Segmentation: Video

Use yolo26n-seg_ncnn_model to predict videos in the ultralytics26 folder (not built-in ultralytics videos).

Enter the code folder:

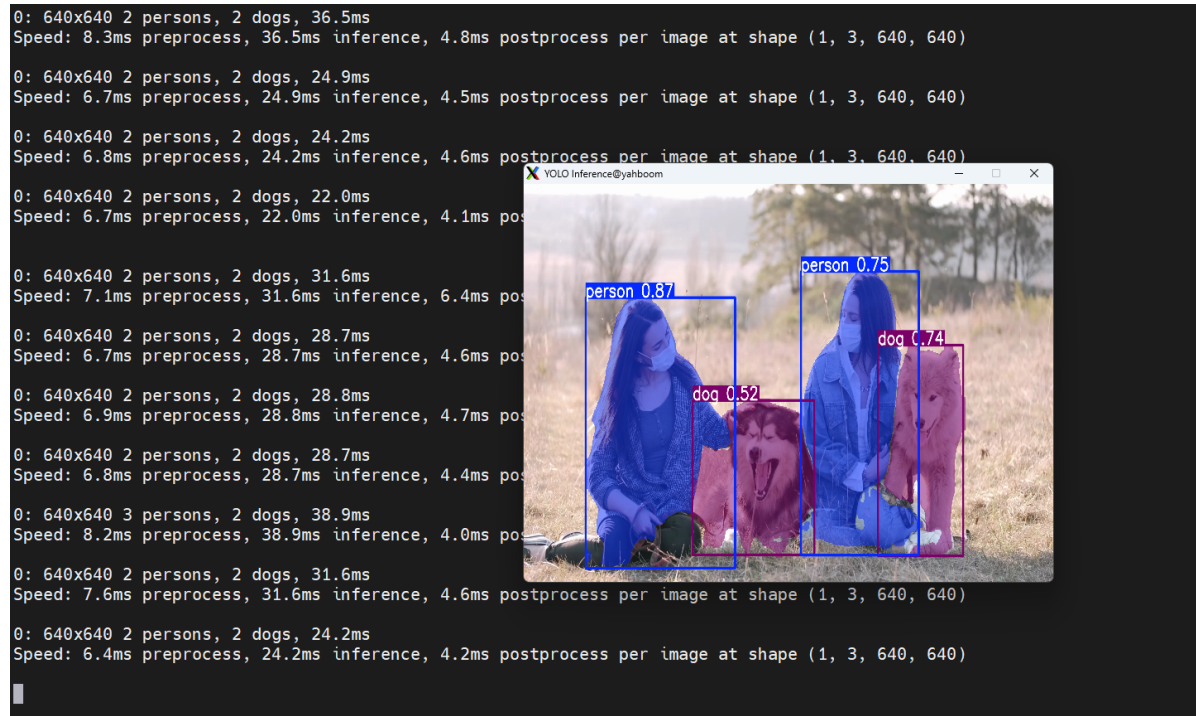
```
cd /home/jetson/ultralytics/yahboom_demo
```

Run the code:

```
python3 02.segmentation_video.py
```

Effect Preview

yolo recognition output video location: /home/jetson/ultralytics/output/



Example code:

```
import cv2
from ultralytics import YOLO

# Load the YOLO model
model = YOLO("/home/jetson/ultralytics/yolo26n-seg.engine")

# Open the video file
video_path = "/home/jetson/ultralytics/ultralytics/videos/people_animals.mp4"
cap = cv2.VideoCapture(video_path)

# Get the video frame size and frame rate
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))

# Define the codec and create a Videowriter object to output the processed video
output_path = "/home/jetson/ultralytics/output/02.people_animals_output.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v') # You can use 'XVID' or 'mp4v'
depending on your platform
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()

    if success:
        # Run YOLO inference on the frame
        results = model(frame)
```

```

# Visualize the results on the frame
annotated_frame = results[0].plot()

# Write the annotated frame to the output video file
out.write(annotated_frame)

# Display the annotated frame
cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

# Break the loop if 'q' is pressed
if cv2.waitKey(1) & 0xFF == ord("q"):
    break
else:
    # Break the loop if the end of the video is reached
    break

# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()

```

4. Instance Segmentation: Real-time Detection

4.1. USB Camera

Use yolo26n-seg_ncnn_model to predict USB camera footage.

Enter the code folder:

```
cd /home/jetson/ultralytics/yahboom_demo
```

Run the code: Click on the preview screen, press q key to terminate the program!

```
python3 02.segmentation_camera_usb.py
```

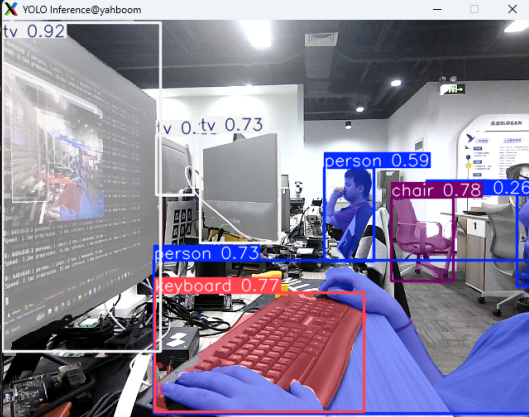
Effect Preview

yolo recognition output video location: /home/jetson/ultralytics/output/

```

Speed: 2.1ms preprocess, 14.1ms inference, 4.3ms postprocess per image at shape (1, 3, 640, 640)
0: 640x640 2 persons, 1 chair, 2 tvs, 1 keyboard, 14.2ms
Speed: 2.0ms preprocess, 14.2ms inference, 4.2ms postprocess per image at shape (1, 3, 640, 640)
0: 640x640 2 persons, 1 chair, 2 tvs, 1 keyboard, 14.7ms
Speed: 2.1ms preprocess, 14.7ms inference, 3.9ms postprocess per image at shape (1, 3, 640, 640)
0: 640x640 2 persons, 1 chair, 2 tvs, 1 keyboard, 14.8ms
Speed: 2.1ms preprocess, 14.8ms inference, 4.0ms postprocess per image at shape (1, 3, 640, 640)
0: 640x640 2 persons, 1 chair, 2 tvs, 1 keyboard, 13.4ms
Speed: 2.2ms preprocess, 13.4ms inference, 4.0ms postprocess per image at shape (1, 3, 640, 640)
0: 640x640 2 persons, 1 chair, 2 tvs, 1 keyboard, 14.0ms
Speed: 2.1ms preprocess, 14.0ms inference, 4.1ms postprocess per image at shape (1, 3, 640, 640)
0: 640x640 2 persons, 1 chair, 2 tvs, 1 keyboard, 14.1ms
Speed: 2.2ms preprocess, 14.1ms inference, 4.2ms postprocess per image at shape (1, 3, 640, 640)
0: 640x640 1 person, 1 chair, 2 tvs, 1 keyboard, 15.2ms
Speed: 2.1ms preprocess, 15.2ms inference, 6.0ms postprocess per image at shape (1, 3, 640, 640)
0: 640x640 2 persons, 1 chair, 2 tvs, 1 keyboard, 15.3ms
Speed: 2.6ms preprocess, 15.3ms inference, 4.7ms postprocess per image at shape (1, 3, 640, 640)
0: 640x640 2 persons, 1 chair, 2 tvs, 1 keyboard, 14.1ms
Speed: 2.6ms preprocess, 14.1ms inference, 4.0ms postprocess per image at shape (1, 3, 640, 640)
0: 640x640 3 persons, 1 chair, 3 tvs, 1 keyboard, 14.3ms
Speed: 2.8ms preprocess, 14.3ms inference, 5.7ms postprocess per image at shape (1, 3, 640, 640)
0: 640x640 3 persons, 1 chair, 2 tvs, 1 keyboard, 14.0ms
Speed: 2.8ms preprocess, 14.0ms inference, 5.8ms postprocess per image at shape (1, 3, 640, 640)

```



Example code:

```

import cv2
from ultralytics import YOLO

# Load the YOLO model
model = YOLO("/home/jetson/ultralytics/yolo26n-seg.engine")

# Open the camera
cap = cv2.VideoCapture(0)
cap.set(6, cv2.VideoWriter_fourcc('M', 'J', 'P', 'G'))
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)

# Get the video frame size and frame rate
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))

# Define the codec and create a VideoWriter object to output the processed video
output_path = "/home/jetson/ultralytics/output/02.segmentation_camera_usb.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v') # You can use 'XVID' or 'mp4v'
depending on your platform
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()

    if success:
        # Run YOLO inference on the frame
        results = model(frame)

        # Visualize the results on the frame
        annotated_frame = results[0].plot()

```

```

# Write the annotated frame to the output video file
out.write(annotated_frame)

# Display the annotated frame
cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

# Break the loop if 'q' is pressed
if cv2.waitKey(1) & 0xFF == ord("q"):
    break
else:
    # Break the loop if the end of the video is reached
    break

# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()

```

4.2. CSI Camera

Use yolo26n-seg_ncnn_model to predict CSI camera footage.

Enter the code folder:

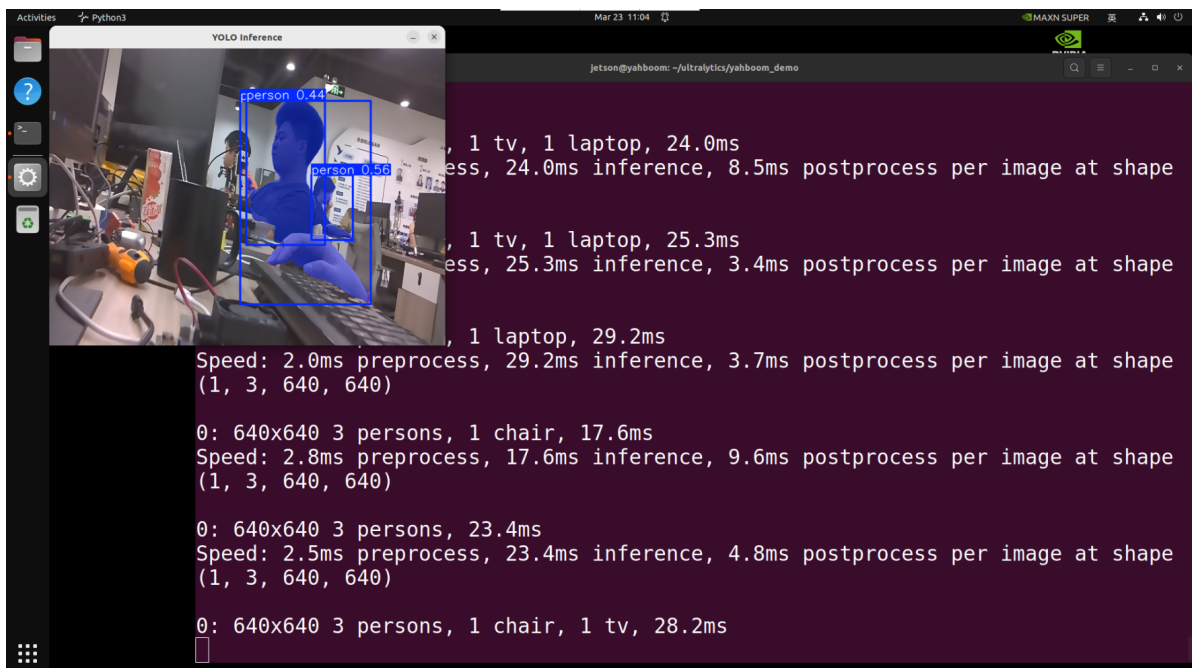
```
cd /home/jetson/ultralytics/yahboom_demo
```

Run the code: Click on the preview screen, press q key to terminate the program!

```
python3 02.segmentation_camera_csi.py
```

Effect Preview

yolo recognition output video location: /home/jetson/ultralytics/output/



Example code:

```
import cv2
```

```

from ultralytics import YOLO
from jetcam.csi_camera import CSICamera

# Load the YOLO model
model = YOLO("/home/jetson/ultralytics/yolo26n-seg.engine")

# Open the camera (CSI Camera)
cap = CSICamera(capture_device=0, width=640, height=480)

# Get the video frame size and frame rate
frame_width = 640
frame_height = 480
fps = 30

# Define the codec and create a Videowriter object to output the processed video
output_path = "/home/jetson/ultralytics/output/02.segmentation_camera_csi.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v') # You can use 'XVID' or 'mp4v'
depending on your platform
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while True:
    # Read a frame from the camera
    frame = cap.read()

    if frame is not None:
        # Run YOLO inference on the frame
        results = model(frame)

        # Visualize the results on the frame
        annotated_frame = results[0].plot()

        # Write the annotated frame to the output video file
        out.write(annotated_frame)

        # Display the annotated frame
        cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

        # Break the loop if 'q' is pressed
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
    else:
        # Break the loop if no frame is received (camera error or end of stream)
        print("No frame received, breaking the loop.")
        break

# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()

```

Reference Materials

[Instance Segmentation - Ultralytics YOLO Documentation](#)

